

Online Retail II

Introduction & Data Dictionary

RFM + K-Means customer segmentation using the **Online Retail II** dataset (UCI ML Repository). The dataset captures 1,067,371 e-commerce transactions over two years (Dec 2009 - Dec 2011) for a UK-based online retailer.

Feature	Type	Description
InvoiceNo	Nominal	6-digit transaction ID. 'C' prefix = cancellation
StockCode	Nominal	5-digit product ID
Description	Nominal	Product name
Quantity	Numeric	Units purchased per transaction
InvoiceDate	Numeric	Transaction timestamp
UnitPrice	Numeric	Price per unit (£)
CustomerID	Nominal	5-digit customer ID
Country	Nominal	Customer country

Goal: Segment customers for targeted CRM strategies.

Source: [UCI Online Retail II](#)

Environment Setup

```
# File handling and data manipulation
import zipfile
import pandas as pd
import numpy as np
```

```
# Data Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Theme
plt.style.use('seaborn-v0_8-whitegrid')
```

Data Wrangling

```
# Load data from ZIP
with zipfile.ZipFile('data.nosync/online+retail+ii.zip','r') as zip:
    with zip.open('online_retail_II.xlsx') as excel_file:
        df_dict = pd.read_excel(excel_file,sheet_name=None)
        retail_bronze = pd.concat(df_dict.values(),axis=0,ignore_index=True)

# Glimpse of the datasets
retail_bronze.head(5)
```

	Invoice	StockCode	Description	Quantity	InvoiceDate
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00

```
print("=="*20)
print("Data Quality Check")
print("=="*20)
print("\nOverview:\n")
print(retail_bronze.info())
print("\n1. Missing Customer ID:")
print(f"    {retail_bronze['Customer ID'].isna().sum()*100/len(retail_bronze):.2f}%")
```

```

print("\n2. Missing Description:")
print(f"    {retail_bronze['Description'].isna().sum()*100/len(retail_bronze):.2f}%")
print("\n3. Duplicates Rows:")
print(f"    {retail_bronze.duplicated(subset=['Invoice', 'StockCode', 'Quantity', 'InvoiceDate',
    'Price', 'Customer ID', 'Country']).sum()*100/len(retail_bronze):.2f}%")
print("\n4. Cancelled Orders:")
print(f"    {retail_bronze['Invoice'].astype(str).str.startswith('C').sum()*100\
/len(retail_bronze):.2f}%")
print("\n5. Zero or Negative Quantity:")
print(f"    {len(retail_bronze[retail_bronze['Quantity']<=0])*100/len(retail_bronze):.2f}%")
print("\n6. Zero or Negative Price:")
print(f"    {len(retail_bronze[retail_bronze['Price']<=0])*100/len(retail_bronze):.2f}%")
print("\n7. Invoice date Range:")
print(f"    {retail_bronze['InvoiceDate'].min()} - {retail_bronze['InvoiceDate'].max()}")

```

```

=====
Data Quality Check
=====

```

Overview:

```

<class 'pandas.DataFrame'>
RangeIndex: 1067371 entries, 0 to 1067370
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Invoice          1067371 non-null  object
1   StockCode       1067371 non-null  object
2   Description     1062989 non-null  object
3   Quantity        1067371 non-null  int64
4   InvoiceDate     1067371 non-null  datetime64[us]
5   Price           1067371 non-null  float64
6   Customer ID    824364 non-null   float64
7   Country         1067371 non-null  str
dtypes: datetime64[us](1), float64(2), int64(1), object(3), str(1)
memory usage: 78.8+ MB
None

```

1. Missing Customer ID:
22.77%

2. Missing Description:

- 0.41%
3. Duplicates Rows:
3.22%
 4. Cancelled Orders:
1.83%
 5. Zero or Negative Quantity:
2.15%
 6. Zero or Negative Price:
0.58%
 7. Invoice date Range:
2009-12-01 07:45:00 - 2011-12-09 12:50:00

Preprocessing Rationale:

1. **Data Hygiene:** Stripped blank spaces in column names to ensure data quality.
2. **Customer ID column:** Dropped missing rows as RFM analysis requires grouping by valid ID. Handled data type by casting floats to integers to strip trailing 0 before converting to strings for use as a key column.
3. **Missing Descriptions:** Retained rows by filling with **unknown** strings. Descriptions not used for clustering in this study.
4. **De-Duplication:** Dropped 3.22% duplicate rows (system errors) that would otherwise inflate results. Kept last occurrence by sorting by **InvoiceDate** before using **keep='last'**.
5. **Dropping Anomalies:** Removed cancelled orders (invoices starting with C) and entries with zero/negative quantities or prices to prevent distortion of Frequency and Monetary calculations.
6. **Feature Engineering:** Calculated **TotalPrice** ($\text{Quantity} \times \text{Price}$) for Monetary calculation.
7. **Pipeline Engineering:** Encapsulated all preprocessing steps into a reusable **cleaning** function for easy debugging and modifications.

```
def cleaning(df):  
    df_clean = df.copy()  
  
    df_clean.columns=df_clean.columns.str.replace(" ", "")
```

```

df_clean.dropna(subset = ["CustomerID"],inplace=True)
df_clean['CustomerID']=df_clean['CustomerID'].astype(int).astype(str)
df_clean['Description'] = df_clean['Description'].fillna('unknown')

df_clean = df_clean.sort_values(by=['InvoiceDate']).\
                drop_duplicates(subset=['Invoice','StockCode',
                                         'Quantity','InvoiceDate',
                                         'Price','CustomerID',
                                         'Country'],keep='last')

anomalies = (df_clean['Invoice'].str.startswith('C')) | \
            (df_clean['Quantity']<=0) | \
            (df_clean['Price']<=0)
df_clean = df_clean[~anomalies]

df_clean['TotalPrice'] = df_clean['Quantity'] * df_clean['Price']

print(f"""\nRows dropped after cleaning:
{(len(df)-len(df_clean))*100/len(df):.2f}%""")
return df_clean.reset_index(drop=True)

retail_silver = cleaning(retail_bronze)

```

Rows dropped after cleaning:
26.98%

Exploratory Data Analysis (EDA)

1. Recency (R): It measures days since last transaction for each customer. As the dataset is historical, a hypothetical snap date was calculated as one day after the last invoice date (2011-12-09). This **snap date** minus each customer's latest Invoice date was used to calculate Recency.
2. Frequency (F): It tracks the number of unique transactions for each customer. Here, **nunique()** transactions are counted because under the same invoice, a customer may have bought different items (Stockcode).
3. Monetary Value (M): This measures how much each customer spends, which is **sum()** of Total Price (Price*Quantity).

```

# Snap date
snap_date = retail_silver['InvoiceDate'].max()+pd.Timedelta(days=1)

# RFM table
rfm = retail_silver.groupby('CustomerID',as_index=False).agg(
    Recency=("InvoiceDate",lambda x: (snap_date-x.max()).days),
    Frequency = ("Invoice","nunique"),
    Monetary = ("TotalPrice","sum")
)

# Descriptive summary
print("=="*20)
print("Base RFM Metrics Summary")
print("=="*20)
print(rfm.describe().round(2))

```

```

=====
Base RFM Metrics Summary
=====

```

	Recency	Frequency	Monetary
count	5878.00	5878.00	5878.00
mean	201.33	6.29	2955.81
std	209.34	13.01	14440.29
min	1.00	1.00	2.95
25%	26.00	1.00	342.28
50%	96.00	3.00	867.74
75%	380.00	7.00	2248.30
max	739.00	398.00	580987.04

```

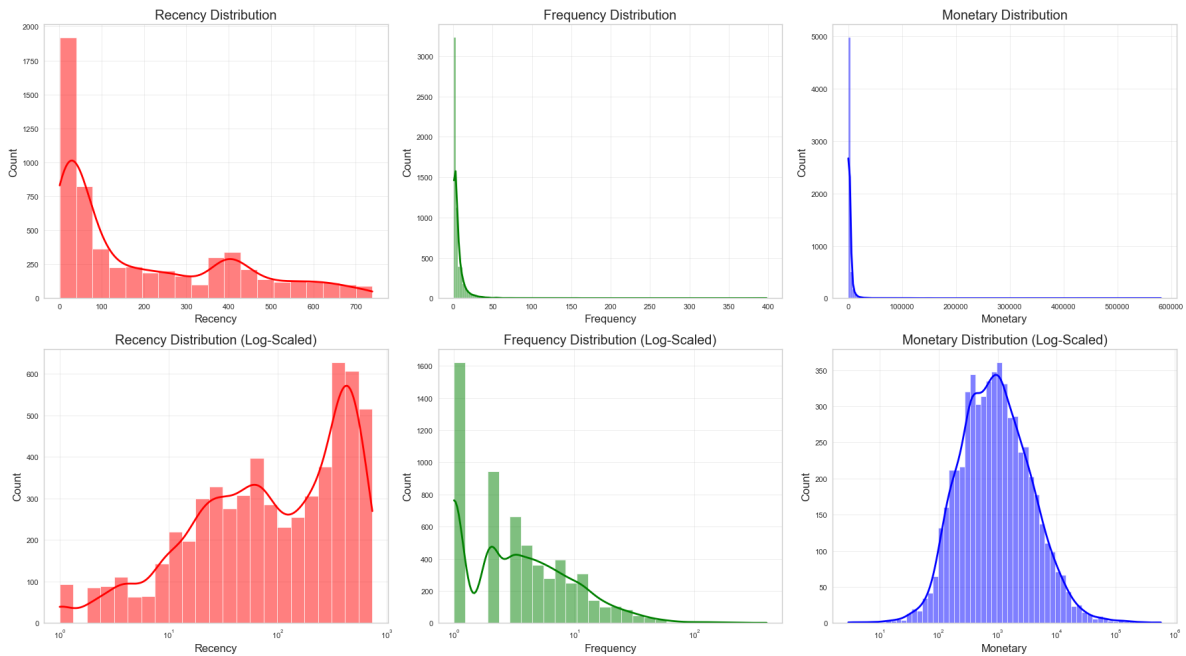
fig,axes = plt.subplots(2,3,figsize=(18,10))
sns.histplot(x="Recency",fill=True,data=rfm,color="red",
    kde=True,ax=axes[0,0])
sns.histplot(x="Frequency",fill=True,data=rfm,color="green",
    kde=True,ax=axes[0,1])
sns.histplot(x="Monetary",fill=True,data=rfm,color="blue",
    kde=True,ax=axes[0,2])
sns.histplot(x="Recency",fill=True,data=rfm,color="red",kde=True,
    log_scale=True,ax=axes[1,0])
sns.histplot(x="Frequency",fill=True,data=rfm,color="green",kde=True,
    log_scale=True,ax=axes[1,1])
sns.histplot(x="Monetary",fill=True,data=rfm,color="blue",

```

```

kde=True,log_scale=True,ax=axes[1,2])
axes[0,0].set_title('Recency Distribution')
axes[0,0].grid(alpha=0.5)
axes[0,1].set_title('Frequency Distribution')
axes[0,1].grid(alpha=0.5)
axes[0,2].set_title('Monetary Distribution')
axes[0,2].grid(alpha=0.5)
axes[1,0].set_title('Recency Distribution (Log-Scaled)')
axes[1,0].grid(alpha=0.5)
axes[1,1].set_title('Frequency Distribution (Log-Scaled)')
axes[1,1].grid(alpha=0.5)
axes[1,2].set_title('Monetary Distribution (Log-Scaled)')
axes[1,2].grid(alpha=0.5)
plt.tight_layout()
plt.show()

```



RFM Segmentation using KMeans Clustering

As observed in the plot and summary description above, RFM table shows massive positive skewness due to large value outliers, such as a £ 580,987.04 maximum spend in Monetary or 398 order count in Frequency.

To uncover true patterns of customer behavior in RFM tables, I will implement K-Means clustering, which is a popular unsupervised machine learning algorithm. K-Means groups data into clusters with naturally meaningful boundaries.

To satisfy the core mathematical assumptions of K-Means, the data needs to be symmetrical and uniformly scaled. I addressed this by applying the two steps below:

1. **Log Transformation(`np.log()`):** Applied across raw RFM table to compress right skewness.
2. **Standardization (`StandardScaler()`):** After the log transform I will scale all R,F and M values to mean = 0 and std= 1 to ensure that larger magnitudes like monetary do not dominates distance calculations.

```
# Log transformation
X=np.log(rfm[["Recency", "Frequency", "Monetary"]])

# train-test split with 80% train set, random state for reproducibility
X_train,X_test = train_test_split(X, test_size=0.2,random_state=42)

# Standardization - fit only on train set to ensure no data leakage
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Transform only on test set
X_test_scaled = scaler.transform(X_test)
```

Validation using Elbow method and Silhouette Coefficient

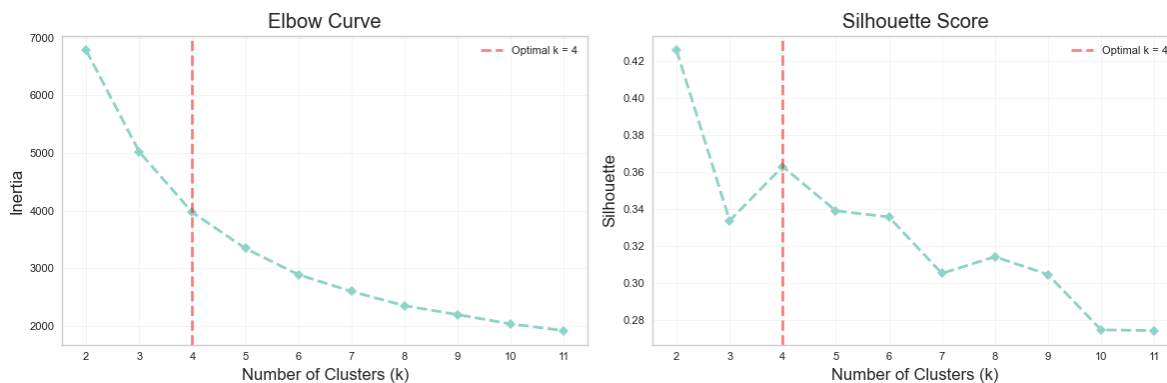
To determine optimal number of clusters, we will use loop to run multiple KMeans models with cluster number(k) in the range of 2 to 12. We will track two main parameters:

1. **Within-Cluster Sum of Squares (WCSS):** This measures intra-cluster distance of data points from its centroid, available by `inertia_` attribute of the fit model. While increasing k will make the WCSS lower, it can also lead to overfitting. For this trade-off, we will use elbow plot to identify optimal cluster number where inertia decreases slowly.
2. **Mean Silhouette Coefficient:** This measures both Inter-cluster separation and nearest-cluster distance ranging from the range -1 to 1. Values near 0 indicate overlapping clusters. This score is calculated on predicted labels to ensure validation on unseen data.


```

# Loops
inertia = []
silhouette = []
for cluster in range(2,12): # Silhouette need at least two cluster.
    kmeans = KMeans(n_clusters = cluster,random_state = 42)
    kmeans.fit(X_train_scaled)
    inertia.append(kmeans.inertia_)
    test_label = kmeans.predict(X_test_scaled) # Predicted labels
    silhouette.append(silhouette_score(X_test_scaled,test_label))
fig,axis = plt.subplots(1,2,figsize=(12,4))
axis[0].plot(range(2,12),inertia,marker='D',linestyle='--')
axis[1].plot(range(2,12),silhouette,marker='D',linestyle='--')
axis[0].set_xlabel("Number of Clusters (k)")
axis[0].set_ylabel("Inertia")
axis[0].set_title("Elbow Curve")
axis[0].set_xticks(range(2,12))
axis[1].set_xticks(range(2,12))
axis[0].axvline(x=4,color='r',linestyle='--',alpha=0.5,label="Optimal k = 4")
axis[0].legend()
axis[1].axvline(x=4,color='r',linestyle='--',alpha=0.5,label="Optimal k = 4")
axis[1].legend()
axis[0].grid(alpha=0.3)
axis[1].grid(alpha=0.3)
axis[1].set_title("Silhouette Score")
axis[1].set_xlabel("Number of Clusters (k)")
axis[1].set_ylabel("Silhouette")
plt.tight_layout()
plt.show()

```



```

ax1.set_xlabel("Number of Clusters (k)")
ax1.set_ylabel("Inertia")
ax1.set_title("Elbow Curve")
ax1.set_xticks(range(2,12))
ax2.set_xticks(range(2,12))
ax1.axvline(x=4,color='red',linestyle='--',alpha=0.5,label="Optimal Cluster = 4")
ax1.legend()
ax2.axvline(x=4,color='red',linestyle='--',alpha=0.5,label="Optimal Cluster = 4")
ax2.legend()
ax1.grid(alpha=0.3)
ax2.grid(alpha=0.3)
ax2.set_title("Silhouette Score")
ax2.set_xlabel("Number of Clusters (k)")
ax2.set_ylabel("Silhouette")
plt.tight_layout()
plt.show()

```

- The elbow curve shows inertia dropping sharply from 7000 to 4000 between $k = 2$ and $k = 4$, then flattens significantly while the silhouette score reaches peak of 0.36 after an initial dip at $k = 4$. Therefore, the optimal cluster, $k = 4$, balances the tight cluster with inter-cluster separation.

```

# Final model fitting using the optimal k

final_kmeans = KMeans(n_clusters = 4, random_state=42)
final_kmeans.fit(X_train_scaled)
final_train_labels = final_kmeans.labels_ # Predicted labels on train dataset
final_test_labels  = final_kmeans.predict(X_test_scaled)

# assign labels back to rfm tables with right index
rfm.loc[X_train.index,'KMeans_Segment'] = final_train_labels
rfm.loc[X_test.index,'KMeans_Segment']  = final_test_labels
rfm['KMeans_Segment'] = rfm['KMeans_Segment'].astype(int)

# Summary
print("=="*25)
print("KMeans Segment Analysis")
print("=="*25)
print(rfm.groupby("KMeans_Segment").agg(
    {"Recency" : "mean",
     "Frequency": "mean",

```

```
"Monetary" : "mean"}
).sort_values('Monetary').round(2))
```

```
=====
KMeans Segment Analysis
=====
```

	Recency	Frequency	Monetary
KMeans_Segment			
1	385.94	1.32	317.49
0	27.78	3.09	843.63
3	233.74	5.06	1896.82
2	26.77	18.94	10565.17

Cluster Profiling & Business Interpretation

While the business logic of interpreting segmentation varies across industries, a standardized mapping logic is applied here to define four segments as below:

1. **Champions (Segment 2):** These customers have the lowest recency level, buy frequently and generate massive revenue (mean: £10,565.17). Marketing should provide exclusive rewards like prime membership, early access to new product launches to maximize their retention.
2. **Promising / New Buyers (Segment 0):** This group has the second best recency but have lower frequency and total spend. They should be the prime targets for advertisement campaigns to turn them into repeat customers.
3. **At Risk (Segment 3):** This group previously generated a modestly high monetary more than once but now on the verge of churning. We need to give them win-back approach like one time discounts.
4. **Churned (Segment 1):** This segment has remained inactive for more than one year with lowest frequency indicating one-time buyers. They should receive low-cost re-engagement efforts only, like cold emails.

```
# Mapping the Segemnt accoring to business interpretation
rfm['KMeans_Segment_name'] = rfm['KMeans_Segment'].map(
    {2: 'Champions',
     0: 'Promising',
     3: 'At Risk',
     1: 'Churned'})
```

```

# 1. Calculate the relative percentage deviation from the overall mean
relative_rfm = (
    rfm[["Recency", "Frequency", "Monetary", "KMeans_Segment_name"]]
    .groupby("KMeans_Segment_name")
    .mean()
    / (rfm[["Recency", "Frequency", "Monetary"]].mean() - 1)
)

# 2. Plot the relative heatmap
plt.figure(figsize=(9, 5))

# Using a diverging colormap (RdBu_r) works best for relative values
# because 0 (the dataset average) will be neutral white/yellow.
sns.heatmap(
    data=relative_rfm,
    annot=True,
    fmt=".2f",
    cmap="RdYlGn_r", # Red for bad, Green for good (or use 'RdBu_r')
    linewidths=0.5,
    center=0, # Forces the colorbar to center its neutral color at 0
)

# 3. Titles and Labels
plt.title(
    "Relative RFM Heatmap: % Deviation from Dataset Average",
    fontsize=13,
    fontweight="bold",
    pad=15,
)
plt.xlabel("RFM Attribute", fontsize=11)
plt.ylabel("Customer Segment", fontsize=11)

plt.tight_layout()
plt.show()

```

```

# Log transformation for plot
rfm_for_plot = np.log(rfm[["Recency", "Frequency", "Monetary"]])

# Standard scaling
rfm_for_plot_scaled = pd.DataFrame(scaler.transform(rfm_for_plot), index=rfm_for_plot.index, columns=rfm_for_plot.columns)

```

```
# Adding the Kmeans level form the rfm table
rfm_for_plot_scaled.loc[rfm.index,"KMeans_Segment_name"]=rfm['KMeans_Segment_name']

# Converting to longer format for plot
rfm_longer =rfm_for_plot_scaled.melt(id_vars="KMeans_Segment_name",
                                     value_vars=["Recency", 'Frequency', 'Monetary'],
                                     var_name="Attribute",
                                     value_name='Matrix')

# plot
sns.lineplot(x="Attribute",y='Matrix',hue='KMeans_Segment_name',estimator="mean",marker="o",
plt.title("Snake Plot: Customer Segments")
plt.xlabel("RFM Attribute", fontsize=12)
plt.ylabel("Standardized Value (Z-Score)",fontsize=12)
plt.axhline(linestyle="--",alpha=0.5,label='Mean')
plt.grid(alpha=0.3)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```

Relative Importance of Segment Attributes (RISA)!

- Silver one is that who just buted but donot develops Frequency or monetay histoy yet.
- Silver need welcomde discount not to be traed linbe same way the gold treated. rgis is the befijt of KMeans.

Cohort Analysis: Time

```
retail_silver['InvoiceMonth']=retail_silver['InvoiceDate'].dt.to_period("M").dt.to_timestamp
retail_silver['CohortMonth']=retail_silver.groupby("CustomerID")["InvoiceMonth"].transform("n
```

```
# Cohort based on month
retail_clean["CohortIndex"]= (retail_clean['InvoiceMonth'].dt.year-retail_clean['InvoiceMonth'].dt.year)
                           (retail_clean['invoice_month'].dt.month-retail_clean['cohort_m
```

- Cohot month fixed that is when they purchased for the first time
- Cohort index how many month since they joined.

Counting monthly active customers in cohorts

```
#wider format
cohots_counts=retail_clean.pivot_table(values="customer_id",
                                       index="cohort_month",
                                       columns='cohort_index',
                                       aggfunc='nunique',
                                       fill_value=0)
cohort_size=cohots_counts.iloc[:,0] # how may custmer at the
retention = cohots_counts.divide(cohort_size,axis=0).round(3)*100
```

```
cohort_size=cohots_counts.iloc[:,0] # how may custmer at the
retention = cohots_counts.divide(cohort_size,axis=0).round(3)*100
retention.index=retention.index.strftime("%Y-%m")
plt.figure(figsize=(25,10))
plt.title('Retention Rate Percentage')
sns.heatmap(data=retention,
            cmap="BuGn",
            annot=True,
            fmt=".0f",
            vmin=0,
            vmax=40,
            linewidths=0.5)
plt.show()
```

- Retained 35% after first month; after two years still 20% so older customer has best loaylty generally have more retention rate
- Retentate stabilize after the first drop in first month.
- columns wise all new customers acquired that month, how many came back just one month later?
- left to right is loyanty or retention
- top to bottom how new customer is adding
- Diagonally occation, chrustmas effect

```
plt.scatter(RFM.iloc[:,3],RFM.iloc[:,1],label=RFM.iloc[:,-1])
```

```
# Check what scores exist in pandas
print("Pandas - Unique Recency scores:")
print(rfm['Recency_score'].unique())
print(rfm['Recency_score'].value_counts().sort_index())

print("\nPandas - Data types:")
print(rfm['Recency_score'].dtype)
```